Driftor

Guia de estudo do repositório driftor-reuso

Manual para a prova de autoria de Desenvolvimento Orientado a Reuso: como o código é organizado, como chega ao banco, os 4 CRUDs, as classes, os 3 padrões de projeto, a variabilidade, a técnica de compactação e as perguntas prováveis.

Equipe
Alander Menezes Arantes de Ávila
Emanuelle Skolut Jose

Referência
github.com/aland3r/driftor-reuso
commit 420f1db · driftor-reuso.fly.dev

Sumário

Como usar este guia

1. O produto

2. A organização do código

3. O caminho de um clique até o banco

4. O banco de dados

5. As classes

6. Os CRUDs

7. Os padrões de projeto

8. Os motores

9. A variabilidade

10. A técnica de compactação de código

11. Perguntas prováveis

12. Checklist para sexta

Como usar este guia

Este manual explica o repositório **driftor-reuso** do começo ao fim, para que os dois integrantes consigam responder qualquer pergunta da prova de autoria, em qualquer CRUD. Os trechos de código são cópias exatas do commit `420f1db`, e o número da linha de cada trecho é o mesmo do GitHub.

Ordem sugerida de estudo:

1. **Capítulos 1 a 3**: o que o produto faz, como o código está organizado e o caminho de um clique até o banco. Sem isso, o resto não se encaixa.
2. **Capítulos 4 a 6**: banco, classes e CRUDs. É o que mais cai.
3. **Capítulos 7 a 10**: padrões de projeto, motores, variabilidade e compactação.
4. **Capítulo 11**: perguntas prováveis. Treinem em dupla, um pergunta e o outro responde sem olhar.

No dia, o material de consulta rápida é a pasta `defesa/` do repositório (índice, CRUDs, padrões, variabilidade, classes e tabelas).

O enunciado, item por item

Item	Pontos	O que pede	Onde está no projeto	Capítulo
01	3,0	Ao menos 10 classes com métodos e atributos	13 classes em `dominio.py`	5
01	(junto)	3 padrões, Opção 02: 2 exemplos de cada	Singleton, Adapter e Observer	7
02	3,0	Mínimo de 2 telas (CRUD) por integrante, ligadas ao banco	Processo, Atividade, Análise e Manutenção	6
02	1,0	A partir de 3 tabelas no banco	7 tabelas em `schema.sql`	4
02	2,0	Um exemplo de como planejaram a variabilidade	Variabilidade por papel: Gestor e Técnico	9
03	1,0	Repositório + técnica de compactação de código focada em reuso, com vídeo	`RepositorioBase` + macros; antes `1f77872`, depois `c13a62b`	10

Regras da prova: a nota é individual, mas se um integrante não souber responder, a equipe também perde. A divisão dos CRUDs, que o enunciado pede (2 por integrante), está no README: Alander com Processo e Atividade, Emanuelle com Análise e Manutenção. Na prova, o professor pode perguntar sobre qualquer um.

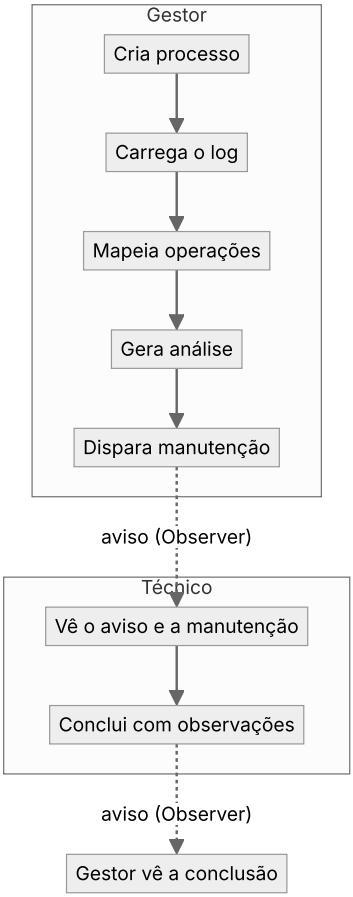
Por que a Opção 02. A Opção 01 exige Singleton (2), Template Method (3) e um terceiro padrão (3). A Opção 02 exige 3 padrões com 2 exemplos de cada. Escolhemos a 02 com Singleton, Adapter e Observer, um de cada família do GoF (criacional, estrutural e comportamental).

1. O produto

O **Driftor** encontra mudanças de comportamento (os *desvios*, ou *drifts*) num processo industrial a partir do log de eventos das máquinas, e transforma cada desvio relevante numa ordem de manutenção.

O fluxo completo, que também é o roteiro da demonstração:

1. O **Gestor** entra no sistema (entrada simulada, sem senha) e cria um **processo** só com o nome, por exemplo "Torno 02".
2. Na página do processo, carrega o **log de eventos** (`.xes` ou `.csv`).
3. **Mapeia** cada operação do log para uma **atividade** do catálogo. Com isso, o processo fica pronto, e aparecem o **grafo** e as **variantes**.
4. Gera uma **análise de desvios**: escolhe a série (a duração de uma atividade em cada trace), o δ (sensibilidade) e o tratamento (bruto ou suavizado). O **ADWIN** encontra os pontos em que a duração mudou.
5. Na análise, o gestor **dispara uma manutenção** a partir de um desvio.
6. O **Técnico de manutenção** entra e vê um **aviso** da manutenção nova. Abre a lista, executa a manutenção e **conclui** escrevendo as observações.
7. O gestor volta e vê o aviso "Manutenção concluída".



Vocabulário. *Log de eventos*: a lista do que cada máquina fez, com início e fim. *Trace*: uma execução completa do processo (um "caso", identificado por `case_id`). *Evento*: uma linha do log dentro de um trace. *Operação*: o nome como veio no log (`rotulo`). *Atividade*: o nome padronizado do catálogo. *Variante*: uma sequência distinta de atividades. *Drift*: um ponto da série em que a média da duração muda.

Dados de teste. A pasta `exemplos/` tem quatro logs sintéticos da empresa fictícia Alfa. Com a série Duração de `Machine_Operating`, δ = 0,002 e tratamento bruto, os resultados são: `ST_20` com 0 desvios, `DR_20` com 4, `DR_MS_20` com 7 e `DR_MS_ST_20` com 9. É o mesmo resultado da Tabela 1 do artigo da pesquisa de ADWIN em que o produto se baseia.

2. A organização do código

O Driftor é um **monólito**: um programa só, em Python, que gera as telas, aplica as regras e fala com o banco. Ele roda num único processo (o servidor `uvicorn`), publicado no Fly.io, com o banco Postgres no Supabase.

```
driftor-reuso/
├── backend/driftor/          tudo o que é avaliado
│   ├── main.py               monta o app, liga os observadores, trata erros
│   ├── dominio.py            as 13 classes de domínio (regras)
│   ├── schema.sql            as 7 tabelas
│   ├── banco.py              BancoDeDados (Singleton): executa o SQL
│   ├── config.py             Configuracao (Singleton) + PAPEIS (variabilidade)
│   ├── telas.py              tela(), ir_para() e exigir() (variabilidade)
│   ├── acesso.py             entrada por perfil (cookie "papel")
│   ├── observador.py         Sujeito, Observador, AvisoNaTela (Observer)
│   ├── repositorio_base.py   RepositorioBase (compactação)
│   ├── mineracao/            leitor_log.py, grafo.py e detector.py (Adapters)
│   ├── processos/            CRUD Processo:   rotas.py + repositorio.py
│   ├── atividades/           CRUD Atividade:  rotas.py + repositorio.py
│   ├── analises/             CRUD Análise:    rotas.py + repositorio.py + eventos.py
│   └── manutencoes/          CRUD Manutenção: rotas.py + repositorio.py + eventos.py
├── frontend/                 templates/ (HTML com Jinja) e static/ (CSS)
├── tests/                    9 testes automáticos
├── exemplos/                 os 4 logs da Alfa
└── defesa/                   material de consulta da prova
```

As camadas

Cada CRUD é uma pasta com dois arquivos. Isso é uma **fatia vertical**: tudo de um assunto fica junto. Dentro da fatia, cada arquivo tem uma responsabilidade:

Camada	Arquivo	Responsabilidade	Exemplo
Rota (controller)	`rotas.py`	recebe a requisição, chama a regra e o repositório, escolhe a próxima tela	`disparar(...)`
Domínio (model)	`dominio.py`	as regras de negócio, sem SQL e sem HTML	`Manutencao.disparar(...)`
Repositório (acesso a dados)	`repositorio.py`	o SQL de cada operação	`inserir(...)` → `insert into manutencoes`
Banco	`banco.py`	conexão e execução do SQL	`BancoDeDados().executar(...)`
Tela (view)	`frontend/templates/`	o HTML	`manutencoes/lista.html`

É o padrão arquitetural **MVC** com uma camada de repositório. A regra nunca conhece o SQL, e o SQL nunca conhece a tela.

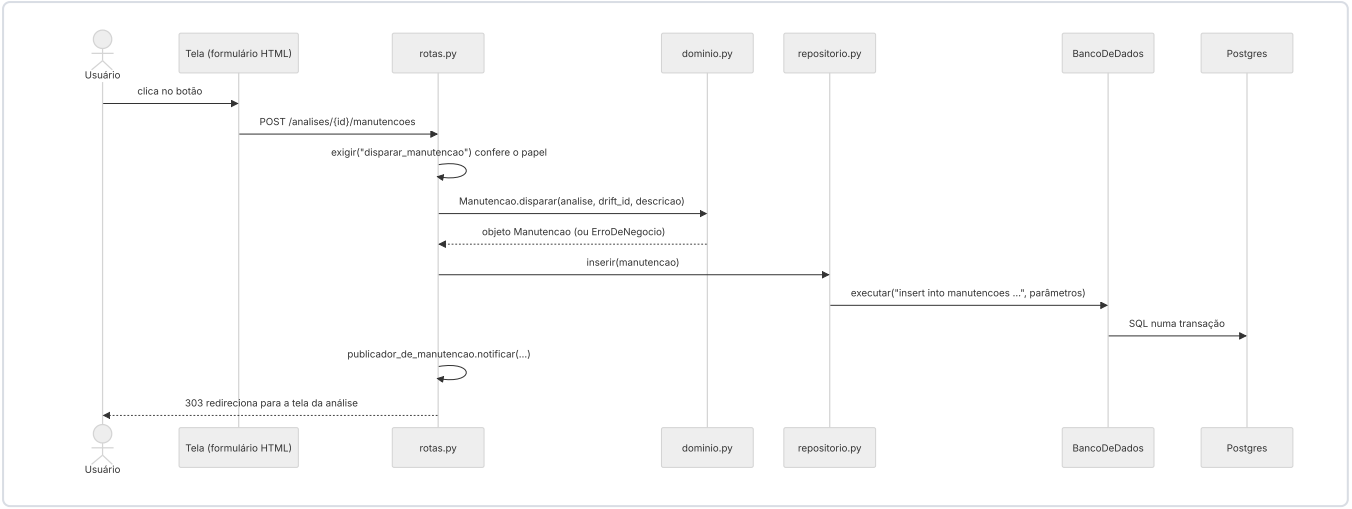
As bibliotecas

Biblioteca	Para quê
FastAPI	servidor web: liga cada URL a uma função (`@rotas.post("/processos")`)
Jinja2	monta o HTML das telas a partir dos templates
SQLAlchemy + psycopg	conexão com o Postgres (pool de conexões) e execução do SQL escrito à mão
PM4Py	mineração de processos: lê o XES e descobre o grafo (DFG)
river	o detector ADWIN
pandas	tabela intermediária para o PM4Py
pytest	testes

Não usamos ORM. O SQLAlchemy é usado só como conexão (`text(sql)`). Todo SQL está escrito à mão nos repositórios, e por isso dá para mostrar o `insert`, o `update` e o `delete` de cada CRUD.

3. O caminho de um clique até o banco

Toda operação de todo CRUD segue o mesmo caminho:



Vamos seguir o exemplo **"Disparar manutenção"** passo a passo.

Passo 1. O botão é um formulário. Não há JavaScript chamando API. O botão envia um formulário HTML com `method="post"` para uma URL:

`frontend/templates/analises/detalhe.html` — linhas 64-72

```
64    <form method="post" action="/analises/{{ analise.id }}/manutencoes" class="form-grade">
65      <label class="campo">Desvio
66        <select class="field" name="drift_id" id="drift_id">
67          {% for d in analise.drifts %}<option value="{{ d.id }}">Desvio {{ d.posicao }}: {{
68  d.resumo() }}</option>{% endfor %}
69        </select>
70      </label>
71      <label class="campo">O que o técnico deve verificar<input class="field" name="descricao"
72  placeholder="Ex.: verificar rolamento do eixo" required></label>
      <div><button class="btn btn-primary">Disparar manutenção</button></div>
    </form>
```

Passo 2. A rota recebe. O FastAPI chama a função ligada a essa URL. O `dependencies=[Depends(exigir(...))]` roda antes e barra quem não tem permissão (variabilidade, capítulo 9). Os campos do formulário chegam como parâmetros (`Form(...)`).

`backend/driftor/manutencoes/rotas.py` — linhas 26-35

```
26  # ---- Create: o gestor dispara a manutenção na tela da análise
27  @rotas.post("/analises/{analise_id}/manutencoes", dependencies=
28  [Depends(exigir("disparar_manutencao"))])
29  def disparar(analise_id: str, drift_id: str = Form(""), descricao: str = Form("")):
30      analise = RepositorioDeAnalises().obter_ou_erro(analise_id)
31      manutencao = Manutencao.disparar(analise, drift_id, descricao)
32      RepositorioDeManutencoes().inserir(manutencao)
33      publicador_de_manutencao.notificar("manutencao_disparada", {  # Observer: avisa o técnico
34          "mensagem": f"Nova manutenção: {manutencao.descricao} (análise “{analise.nome}”).",
35      })
        return ir_para(f"/analises/{analise_id}")
```

Passo 3. A regra. `Manutencao.disparar` valida e cria o objeto. Se algo estiver errado, lança `ErroDeNegocio`, e o `main.py` transforma o erro numa tela com status 422.

backend/driftor/dominio.py — linhas 338-344

```
338     def disparar(analise: Analise, drift_id: str | None, descricao: str) -> Manutencao:
339         if drift_id and drift_id not in {d.id for d in analise.drifts}:
340             raise ErroDeNegocio("Este desvio não pertence à análise.")
341         descricao = (descricao or "").strip()
342         if not descricao:
343             raise ErroDeNegocio("Descreva o que o técnico deve verificar.")
344         return Manutencao(novo_id(), analise.id, drift_id or None, descricao)
```

Passo 4. O SQL. O repositório escreve o `insert` com **parâmetros nomeados** (`:id`, `:descricao`). Os valores nunca são colados no texto do SQL, o que impede *SQL injection*.

backend/driftor/manutencoes/repositorio.py — linhas 41-48

```
41      # ---- Create
42      def inserir(self, manutencao: Manutencao) -> None:
43          self.banco.executar(
44              """insert into manutencoes (id, analise_id, drift_id, descricao)
45                 values (:id, :analise_id, :drift_id, :descricao)""",
46              id=manutencao.id, analise_id=manutencao.analise_id,
47              drift_id=manutencao.drift_id, descricao=manutencao.descricao,
48          )
```

Passo 5. A execução. O `BancoDeDados` abre uma transação (`engine.begin()`), executa e confirma. Se der erro, desfaz tudo.

backend/driftor/banco.py — linhas 30-42

```
30      def consultar(self, sql: str, **parametros) -> list[dict]:
31          """SELECT: devolve as linhas como dicionários."""
32          with self.engine.connect() as conexao:
33              return [dict(linha._mapping) for linha in conexao.execute(text(sql), parametros)]
34
35      def consultar_um(self, sql: str, **parametros) -> dict | None:
36          linhas = self.consultar(sql, **parametros)
37          return linhas[0] if linhas else None
38
39      def executar(self, sql: str, **parametros) -> int:
40          """INSERT, UPDATE ou DELETE numa transação. Devolve quantas linhas mudaram."""
41          with self.engine.begin() as conexao:
42              return conexao.execute(text(sql), parametros).rowcount
```

Passo 6. A resposta. Depois de um POST, a rota responde **303** e manda o navegador abrir outra URL (`ir_para`). É o padrão **Post/Redirect/Get**: recarregar a página não repete o envio.

backend/driftor/telas.py — linhas 44-46

```
44  def ir_para(url: str) -> RedirectResponse:
45      """Depois de um POST, volta para uma tela (padrão Post/Redirect/Get)."""
46      return RedirectResponse(url, status_code=303)
```

Os erros. Três exceções viram telas, em `main.py`: `PrecisaEntrar` (sem perfil: vai para a entrada), `SemPermissao` (403) e `ErroDeNegocio` (422).

backend/driftor/main.py — linhas 62-78

```python
@app.exception_handler(PrecisaEntrar)
def precisa_entrar(_request: Request, _erro: PrecisaEntrar):
    return ir_para("/entrar")


@app.exception_handler(SemPermissao)
def sem_permissao(request: Request, _erro: SemPermissao):
    resposta = tela(request, "erro.html", mensagem="Este perfil não tem acesso a esta tela.",
voltar="/")
    resposta.status_code = 403
    return resposta


@app.exception_handler(ErroDeNegocio)
def erro_de_negocio(request: Request, erro: ErroDeNegocio):
    resposta = tela(request, "erro.html", mensagem=str(erro),
voltar=request.headers.get("referer", "/"))
    resposta.status_code = 422
    return resposta
```

4. O banco de dados

Onde fica e como conecta

- **Banco:** Postgres 17 no Supabase, num **schema próprio**, `driftor_reuso`, separado do resto do projeto.
- **Usuário próprio:** a role `driftor_reuso`, dona do schema. O app não usa o usuário `postgres`. Isso é o princípio do **menor privilégio**.
- **Endereço:** a variável de ambiente `DATABASE_URL`, no formato `postgresql://driftor_reuso.<projeto>:<senha>@aws-1-sa-east-1.pooler.supabase.com:5432/postgres`. A senha fica só no secret do GitHub e no Fly, nunca no código.
- **Pooler:** a conexão passa pelo *Session pooler* do Supabase (porta 5432), que reaproveita conexões.
- **Schema certo sem escrever o nome:** como a role se chama `driftor_reuso` e o Postgres procura primeiro o schema com o nome do usuário (`search_path` = `"$user", public`), as tabelas são criadas e lidas em `driftor_reuso` automaticamente.

Quem lê o endereço é a `Configuracao` (Singleton). Quem abre a conexão é o `BancoDeDados` (Singleton):

backend/driftor/banco.py — linhas 20-28

```
20  class BancoDeDados:
21      _instancia: BancoDeDados | None = None
22
23      def __new__(cls) -> BancoDeDados:
24          if cls._instancia is None:
25              cls._instancia = super().__new__(cls)
26              url = Configuracao().url_do_banco.replace("postgresql://", "postgresql+psycopg://", 1)
27              cls._instancia.engine: Engine = create_engine(url, pool_pre_ping=True, connect_args=
28  {"connect_timeout": 10})
          return cls._instancia
```

- `create_engine` cria um **pool** de conexões: elas ficam abertas e são reaproveitadas, em vez de abrir uma nova a cada clique.
- `pool_pre_ping=True` testa a conexão antes de usar (o pooler pode fechar conexões paradas).
- `connect_timeout=10` desiste depois de 10 segundos se o banco não responder.

Como as tabelas nascem

Não existe ferramenta de migração. Quando o app sobe, a função `ao_subir` (o *lifespan* do FastAPI) executa o arquivo `schema.sql` inteiro. Como cada comando usa `create table if not exists`, rodar de novo não estraga nada.

backend/driftor/main.py — linhas 25-30

```
25  async def ao_subir(_app: FastAPI):
26      try:
27          BancoDeDados().aplicar_schema()  # cria as tabelas do schema.sql, se ainda não existem
28      except Exception as erro:  # o app sobe mesmo assim; /saude/banco mostra o motivo
29          print(f"banco indisponível ao subir: {type(erro).__name__}: {erro}", flush=True)
30      yield
```

A rota `/saude/banco` aplica o schema e conta as tabelas. O deploy chama essa rota no fim e só fica verde quando ela responde `{"ok": true, "tabelas": 7}`.

As 7 tabelas

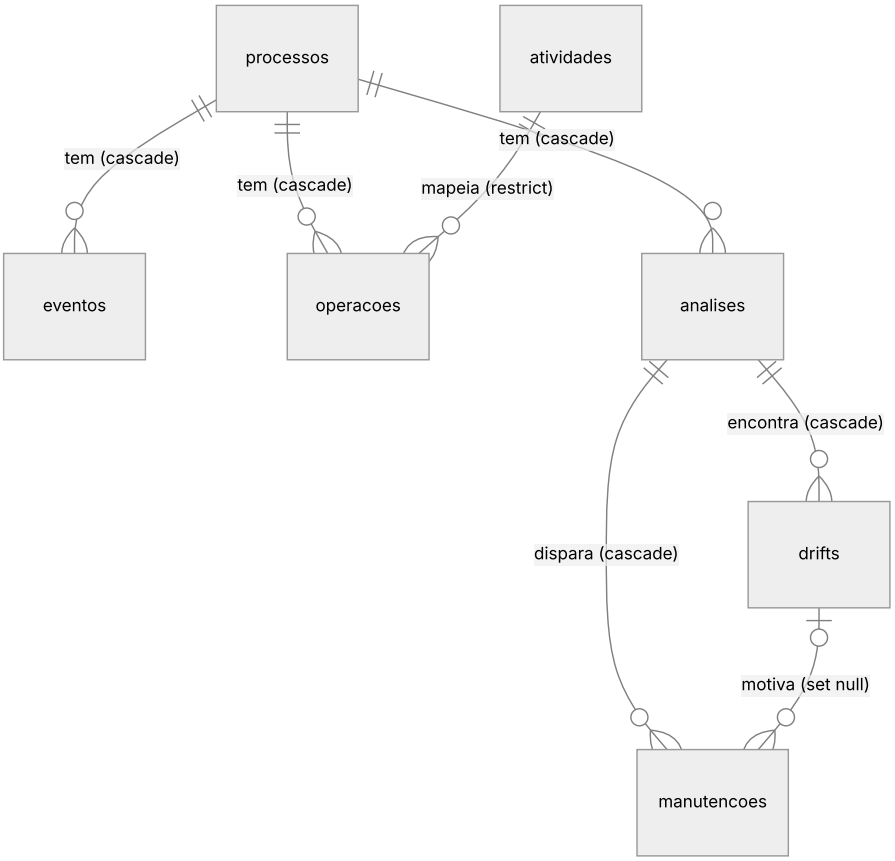


Tabela	Guarda	CRUD
`processos`	nome, status (vazio, mapeando, pronto), arquivo, totais	Processo
`eventos`	cada linha do log: trace, ordem, rótulo, início, duração	Processo
`atividades`	o catálogo: nome único (sem diferenciar maiúsculas) e descrição	Atividade
`operacoes`	cada rótulo do log, quantas vezes aparece e a atividade ligada	Processo
`analises`	parâmetros (δ, tratamento, série), totais e a série em JSON	Análise
`drifts`	cada desvio: posição, índice, início, médias antes e depois, variação	Análise
`manutencoes`	descrição, status (pendente, concluída), observações, datas	Manutenção

As regras de exclusão das chaves estrangeiras (caem muito em pergunta):

- `on delete cascade`: excluir o pai apaga os filhos. Excluir um processo apaga eventos, operações e análises; excluir uma análise apaga drifts e manutenções.
- `on delete restrict`: o banco **recusa** excluir o pai enquanto houver filhos. Uma atividade usada em alguma operação não pode ser excluída. O repositório captura o erro (`IntegrityError`) e mostra a

mensagem.

- `on delete set null`: se o drift sumir, a manutenção fica, mas sem o drift.

backend/driftor/schema.sql linhas 29-44

```sql
-- CRUD de Atividade: catálogo de atividades
create table if not exists atividades (
    id        uuid primary key,
    nome      text not null,
    descricao text not null default ''
);
create unique index if not exists atividades_nome_unico on atividades (lower(nome));

-- Liga cada operação do log a uma atividade do catálogo (mapeamento)
create table if not exists operacoes (
    processo_id  uuid not null references processos(id) on delete cascade,
    rotulo       text not null,
    ocorrencias  integer not null,
    casos        integer not null,
    atividade_id uuid references atividades(id) on delete restrict,
    primary key (processo_id, rotulo)
```

Índices: `atividades_nome_unico` em `lower(nome)` impede duas atividades "Corte" e "corte"; `eventos_processo` acelera a leitura dos eventos de um processo, na ordem.

5. As classes

Orientação a objetos em Python, para quem pensa em Java

Conceito	Java	Python (no nosso código)
Classe	`public class Processo { ... }`	`class Processo:`
Atributo	`private String nome;`	`nome: str` (com `@dataclass`)
Construtor	`public Processo(String nome)`	gerado pelo `@dataclass`, ou `__init__`
Método	`public void renomear(String n)`	`def renomear(self, novo_nome: str) -> None:`
`this`	`this.nome`	`self.nome`
Método estático (fábrica)	`public static Processo criar(...)`	`@staticmethod def criar(...)`
Herança	`class A extends B`	`class A(B):`
Interface / classe abstrata	`interface X` / `abstract class X`	`class X(ABC):` com `@abstractmethod`
Genérico	`class Repo<T>`	`class RepositorioBase(Generic[T])`
Singleton	construtor privado + `getInstance()`	`__new__` que devolve sempre `_instancia`
Tipo	obrigatório	*type hints* (`-> list[Trace]`): documentam, não obrigam

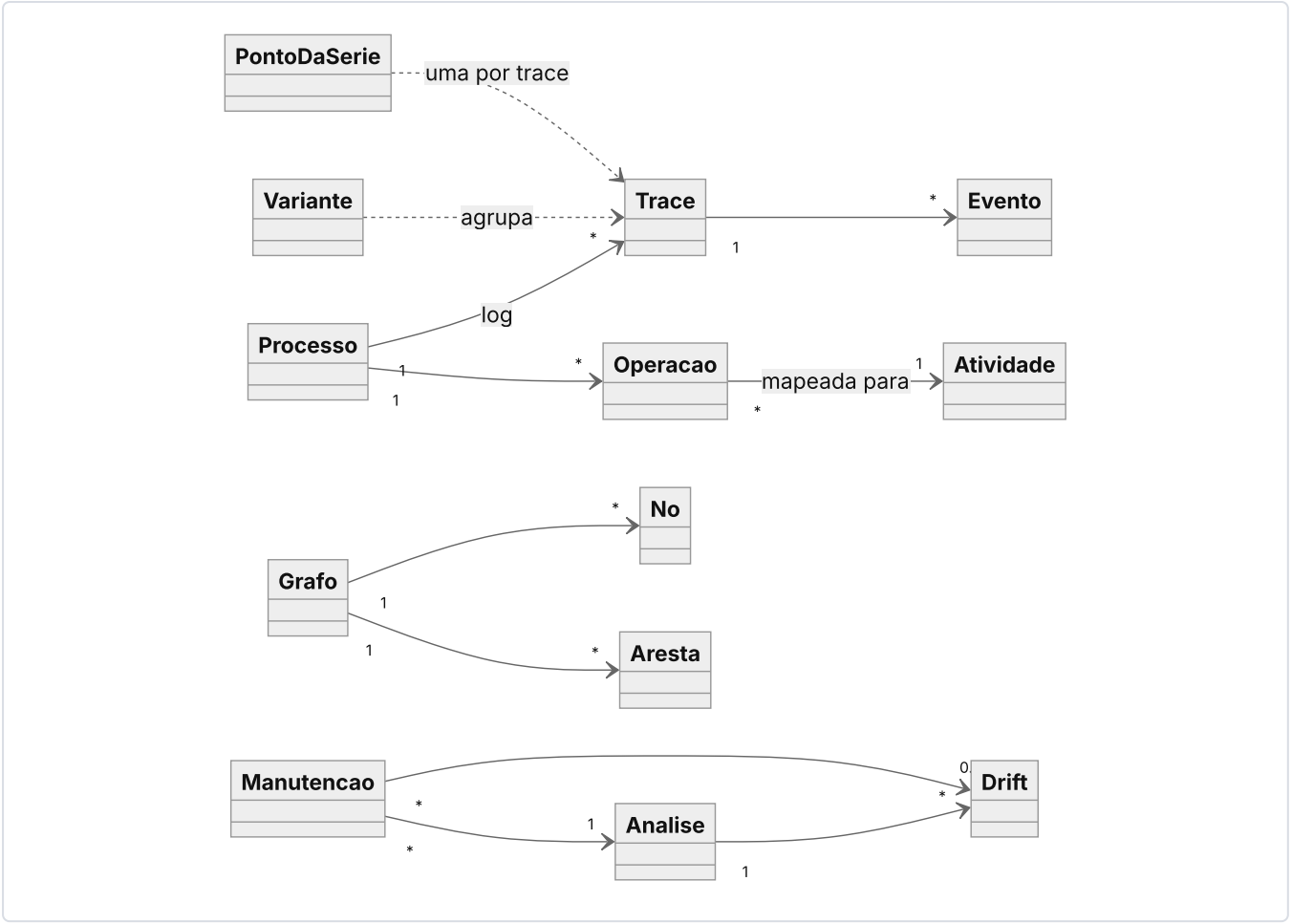
`@dataclass` gera sozinho o construtor, a comparação e a impressão a partir dos atributos declarados. É o equivalente a um *record* ou a uma classe com construtor e getters. `self` é o próprio objeto e é sempre o primeiro parâmetro de um método. **Encapsulamento** em Python é por convenção: um atributo que começa com `_` (como `_instancia` e `_observadores`) é interno.

As 13 classes de domínio

Todas em `backend/driftor/dominio.py`.

Classe	Atributos	Métodos	O que representa
`Processo` `dominio.py:33`	nome, status, arquivo, formato, total_traces, total_eventos	`criar`, `renomear`, `carregar_log`, `marcar_pronto`, `pode_analisar`	o processo de uma máquina
`Evento` `dominio.py:76`	rotulo, inicio, duracao_s, atividade	`nome_no_grafo`	uma linha do log
`Trace` `dominio.py:87`	case_id, eventos	`inicio`, `sequencia`, `duracao`	uma execução completa

Classe	Atributos	Métodos	O que representa
`Atividade` dominio.py:110	nome, descricao	`criar`, `editar`	item do catálogo
`Operacao` dominio.py:131	rotulo, ocorrencias, casos, atividade	`contar`, `sugerir_atividade`, `mapear`	rótulo do log e sua atividade
`Variante` dominio.py:170	numero, sequencia, total_traces	`agrupar`, `descricao`	um caminho distinto
`No` dominio.py:192	atividade, frequencia, duracao_media_s	`rotulo`	nó do grafo
`Aresta` dominio.py:202	origem, destino, frequencia	(só dados)	passagem entre atividades
`Grafo` dominio.py:209	nos, arestas	`montar`, `para_mermaid`	o mapa do processo
`PontoDaSerie` dominio.py:244	posicao, case_id, valor	`serie_de_duracao`	um valor da série analisada
`Drift` dominio.py:263	posicao, indice, inicio_anomalia, media_antes, media_depois, variacao_pct	`aumentou`, `resumo`	um desvio encontrado
`Analise` dominio.py:281	nome, atividade, delta, tratamento, total_observacoes, drifts	`criar`, `renomear`, `registrar_resultado`	uma rodada do ADWIN
`Manutencao` dominio.py:325	analise_id, drift_id, descricao, status, observacoes, datas	`disparar`, `esta_pendente`, `editar`, `concluir`	ordem de manutenção



As regras ficam nos métodos. Por exemplo, o processo só aceita análise depois de mapeado, e o método que diz isso é dele:

backend/driftor/dominio.py — linhas 33-70

```python
class Processo:
    id: str
    nome: str
    status: str = "vazio"          # vazio → mapeando → pronto
    arquivo: str | None = None
    formato: str | None = None
    total_traces: int = 0
    total_eventos: int = 0
    criado_em: datetime | None = None

    @staticmethod
    def criar(nome: str) -> Processo:
        nome = limpar_nome(nome)
        if not nome:
            raise ErroDeNegocio("Dê um nome ao processo.")
        return Processo(id=novo_id(), nome=nome)

    def renomear(self, novo_nome: str) -> None:
        novo_nome = limpar_nome(novo_nome)
        if not novo_nome:
            raise ErroDeNegocio("O nome não pode ficar vazio.")
        self.nome = novo_nome

    def carregar_log(self, arquivo: str, formato: str, traces: list[Trace]) -> None:
        """Um log novo substitui o anterior e o processo volta a precisar de mapeamento."""
        if not traces:
            raise ErroDeNegocio("O log não tem nenhum trace.")
        self.arquivo = arquivo
        self.formato = formato
        self.total_traces = len(traces)
        self.total_eventos = sum(len(t.eventos) for t in traces)
        self.status = "mapeando"

    def marcar_pronto(self) -> None:
        self.status = "pronto"

    def pode_analisar(self) -> bool:
        return self.status == "pronto"
```

Fábricas (`criar`, `disparar`). Em vez de chamar o construtor direto, as rotas chamam um método estático que valida antes de criar. Assim, um objeto inválido nunca existe.

As outras classes

Classe	Papel
`Configuracao`, `BancoDeDados`	Singletons (capítulo 7)
`RepositorioBase` e os 4 repositórios	acesso ao banco (capítulos 6 e 10)
`MineradorDeGrafo`, `Pm4pyMineradorAdapter`, `DetectorDeDrift`, `RiverAdwinAdapter`	Adapters (capítulo 7)
`Sujeito`, `Observador`, `AvisoNaTela`, `AtualizadorDoProcesso`	Observer (capítulo 7)
`ErroDeNegocio`, `PrecisaEntrar`, `SemPermissao`	exceções que viram telas de erro

6. Os CRUDs

São 4 CRUDs, todos com as 4 operações e todos ligados ao banco. Todos os caminhos ficam dentro de `backend/driftor/`.

O que vem herdado de `RepositorioBase`: `listar`, `obter`, `obter_ou_erro` e `excluir`. Por isso, alguns Read e Delete não têm SQL no repositório do próprio CRUD: o SQL está na base, com o nome da tabela vindo do atributo `tabela` da filha.

backend/driftor/repositorio_base.py linhas 18-47

```python
class RepositorioBase(Generic[T]):
    tabela: str = ""
    colunas: str = "*"
    ordem: str = "id"
    nome_no_singular: str = "registro"

    def __init__(self) -> None:
        self.banco = BancoDeDados()

    def criar_objeto(self, linha: dict) -> T:
        raise NotImplementedError

    # ---- Read
    def listar(self) -> list[T]:
        linhas = self.banco.consultar(f"select {self.colunas} from {self.tabela} order by
{self.ordem}")
        return [self.criar_objeto(linha) for linha in linhas]

    def obter(self, id: str) -> T | None:
        linha = self.banco.consultar_um(f"select {self.colunas} from {self.tabela} where id =
:id", id=id)
        return self.criar_objeto(linha) if linha else None

    def obter_ou_erro(self, id: str) -> T:
        objeto = self.obter(id)
        if objeto is None:
            raise ErroDeNegocio(f"{self.nome_no_singular.capitalize()} não encontrado(a).")
        return objeto

    # ---- Delete
    def excluir(self, id: str) -> None:
        self.banco.executar(f"delete from {self.tabela} where id = :id", id=id)
```

Processo (`processos/`)

Operação	Requisição	Rota → regra → SQL
Create	`POST /processos`	`criar` → `Processo.criar` → `insert into processos`
Read	`GET /processos`, `GET /processos/{id}`	`listar`, `ver` → `Grafo.montar`, `Variante.agrupar` → `select` (base) e `traces`
Update	`POST /processos/{id}/log`	`carregar_log` → `Processo.carregar_log` → `substituir_log` (apaga e grava eventos e operações)

Operação	Requisição	Rota → regra → SQL
Update	`POST /processos/{id}/operacoes`	`salvar_operacoes` → `Operacao.mapear`, `marcar_pronto` → `salvar_mapeamento`, `atualizar`
Update	`POST /processos/{id}/renomear`	`renomear` → `Processo.renomear` → `update processos`
Delete	`POST /processos/{id}/excluir`	`excluir` → `delete from processos` (cascade)

Os Updates do log e do mapeamento são o coração do fluxo. O mapeamento liga cada operação a uma atividade; se o nome digitado não existir no catálogo, ele entra no catálogo na hora:

backend/driftor/processos/rotas.py — linhas 77-94

```python
@rotas.post("/processos/{id}/operacoes")
async def salvar_operacoes(request: Request, id: str):
    repo = RepositorioDeProcessos()
    processo = repo.obter_ou_erro(id)
    formulario = await request.form()
    atividades = RepositorioDeAtividades()
    ligacoes: dict[str, str] = {}
    for operacao in repo.operacoes(id):
        operacao.mapear(str(formulario.get(f"op::{operacao.rotulo}", "")))
        atividade = atividades.obter_por_nome(operacao.atividade)
        if atividade is None:  # nome novo entra no catálogo
            atividade = Atividade.criar(operacao.atividade)
            atividades.inserir(atividade)
        ligacoes[operacao.rotulo] = atividade.id
    repo.salvar_mapeamento(id, ligacoes)
    processo.marcar_pronto()
    repo.atualizar(processo)
    return ir_para(f"/processos/{id}")
```

Atividade (`atividades/`)

Operação	Requisição	Rota → regra → SQL
Create	`POST /atividades`	`criar` → `Atividade.criar` → `insert into atividades`
Read	`GET /atividades`	`listar` → `select` (base)
Update	`POST /atividades/{id}`	`salvar` → `Atividade.editar` → `update atividades`
Delete	`POST /atividades/{id}/excluir`	`excluir` → `delete` (base), recusado pelo `restrict` se a atividade estiver em uso

O Delete sobrescreve o da base para traduzir o erro do banco:

backend/driftor/atividades/repositorio.py — linhas 48-53

```python
    # ---- Delete (o banco recusa se algum processo usa a atividade: on delete restrict)
    def excluir(self, id: str) -> None:
        try:
            super().excluir(id)
        except IntegrityError as erro:
            raise ErroDeNegocio("Esta atividade está mapeada em um processo e não pode ser excluída.") from erro
```

Análise (`analises/`)

Operação	Requisição	Rota → regra → SQL
Create	`POST /processos/{id}/analises`	`criar` → `Analise.criar`, ADWIN, `registrar_resultado` → `insert into analises` e `drifts`
Read	`GET /analises`, `GET /analises/{id}`	`listar`, `ver` → `listar_resumo` (join com processos) e `obter_ou_erro`
Update	`POST /analises/{id}/renomear`	`renomear` → `Analise.renomear` → `update analises`
Delete	`POST /analises/{id}/excluir`	`excluir` → `delete from analises` (cascade)

O Create da análise é o que mais usa padrões: o Adapter do ADWIN e o Observer.

backend/driftor/analises/rotas.py linhas 29-56

```
29  # ---- Create: botão "Gerar análise de desvios" na página do processo
30  @rotas.post("/processos/{processo_id}/analises", dependencies=so_gestor)
31  def criar(
32      processo_id: str,
33      nome: str = Form(""),
34      atividade: str = Form(""),
35      delta: float = Form(0.002),
36      tratamento: str = Form("bruto"),
37      detector: DetectorDeDrift = Depends(detector),
38  ):
39      processos = RepositorioDeProcessos()
40      processo = processos.obter_ou_erro(processo_id)
41      analise = Analise.criar(processo, nome, delta, tratamento, atividade)
42
43      pontos = PontoDaSerie.serie_de_duracao(processos.traces(processo_id), analise.atividade)
44      if len(pontos) < 2:
45          raise ErroDeNegocio("A série tem menos de duas observações.")
46      valores = [p.valor for p in pontos]
47      if tratamento == "tratado":
48          valores = suavizar(valores)
49      analise.registrar_resultado(valores, detector.detectar(valores, delta))  # Adapter em uso
50
51      RepositorioDeAnalises().inserir(analise, valores)
52      publicador_de_analise.notificar("analise_concluida", {  # Observer em uso
53          "processo_id": processo_id,
54          "mensagem": f"Análise “{analise.nome}” concluída: {len(analise.drifts)} desvio(s)
55  encontrado(s).",
56      })
        return ir_para(f"/analises/{analise.id}")
```

O `insert` grava a análise e todos os desvios na mesma transação (`executar_varios` grava várias linhas de uma vez):

backend/driftor/analises/repositorio.py — linhas 45-60

```
    # ---- Create
    def inserir(self, analise: Analise, serie: list[float]) -> None:
        self.banco.executar(
            """insert into analises (id, processo_id, nome, atividade, delta, tratamento,
                                     total_observacoes, total_drifts, serie)
               values (:id, :processo_id, :nome, :atividade, :delta, :tratamento,
                       :total_observacoes, :total_drifts, :serie)""",
            id=analise.id, processo_id=analise.processo_id, nome=analise.nome,
atividade=analise.atividade,
            delta=analise.delta, tratamento=analise.tratamento,
total_observacoes=analise.total_observacoes,
            total_drifts=len(analise.drifts), serie=json.dumps(serie),
        )
        self.banco.executar_varios(
            """insert into drifts (id, analise_id, posicao, indice, inicio_anomalia, media_antes,
media_depois, variacao_pct)
               values (:id, :analise_id, :posicao, :indice, :inicio_anomalia, :media_antes,
:media_depois, :variacao_pct)""",
            [{**vars(d), "analise_id": analise.id} for d in analise.drifts],
        )
```

Manutenção (`manutencoes/`)

Operação	Requisição	Quem	Rota → regra → SQL
Create	`POST /analises/{id}/manutencoes`	Gestor	`disparar` → `Manutencao.disparar` → `insert into manutencoes`
Read	`GET /manutencoes`, `GET /manutencoes/{id}`	os dois	`listar`, `ver` → `listar_resumo` (join) e `obter_ou_erro`
Update	`POST /manutencoes/{id}/concluir`	Técnico	`concluir` → `Manutencao.concluir` → `update manutencoes`
Update	`POST /manutencoes/{id}`	Gestor	`editar` → `Manutencao.editar` → `update manutencoes`
Delete	`POST /manutencoes/{id}/excluir`	Gestor	`excluir` → `delete from manutencoes` (base)

backend/driftor/manutencoes/rotas.py — linhas 47-57

```
# ---- Update pelo técnico: concluir com observações
@rotas.post("/manutencoes/{id}/concluir", dependencies=[Depends(exigir("concluir_manutencao"))])
def concluir(id: str, observacoes: str = Form("")):
    repo = RepositorioDeManutencoes()
    manutencao = repo.obter_ou_erro(id)
    manutencao.concluir(observacoes)
    repo.atualizar(manutencao)
    publicador_de_manutencao.notificar("manutencao_concluida", {  # Observer: avisa o gestor
        "mensagem": f"Manutenção concluída pelo técnico: {manutencao.descricao}.",
    })
    return ir_para("/manutencoes")
```

backend/driftor/manutencoes/repositorio.py — linhas 23-36

```
    # ---- Read: a lista com o nome da análise, do processo e o número do desvio
    def listar_resumo(self, analise_id: str | None = None) -> list[dict]:
        filtro = "where m.analise_id = :analise_id" if analise_id else ""
        return self.banco.consultar(
            f"""select m.id, m.descricao, m.status, m.observacoes, m.criado_em, m.concluida_em,
                       a.id as analise_id, a.nome as analise, p.nome as processo, d.posicao as
desvio
                from manutencoes m
                join analises a on a.id = m.analise_id
                join processos p on p.id = a.processo_id
                left join drifts d on d.id = m.drift_id
                {filtro}
                order by (m.status = 'pendente') desc, m.criado_em desc""",
            analise_id=analise_id,
        )
```

O `listar_resumo` usa `join` para trazer, numa consulta só, o nome da análise, do processo e o número do desvio. O `order by (m.status = 'pendente') desc` coloca as pendentes primeiro.

7. Os padrões de projeto

Opção 02: 3 padrões, 2 exemplos de cada. O enunciado pede padrões **focados em reuso**, e cada um reaproveita algo de um jeito diferente: o Singleton reaproveita um objeto, o Adapter reaproveita uma biblioteca de terceiros, e o Observer reaproveita quem publica um evento.

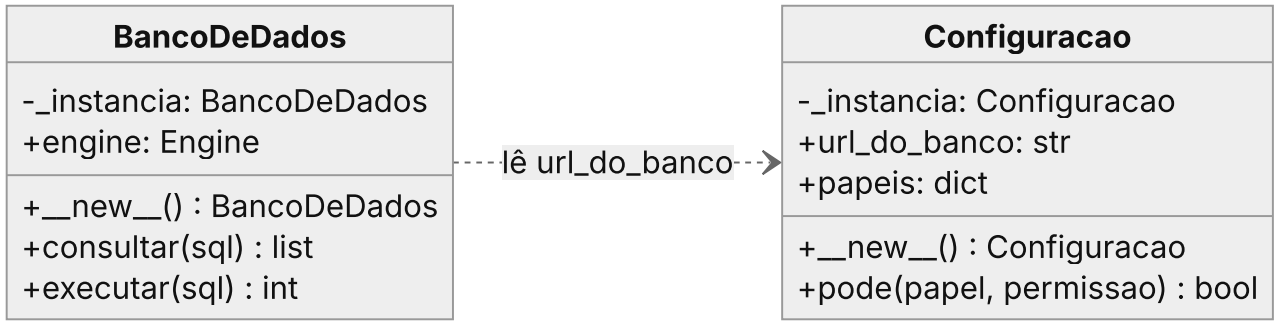
Padrão	Família	Exemplo 1	Exemplo 2
Singleton	criacional	`Configuracao`	`BancoDeDados`
Adapter	estrutural	`Pm4pyMineradorAdapter`	`RiverAdwinAdapter`
Observer	comportamental	análise concluída	manutenção disparada e concluída

7.1 Singleton

Problema. Algumas coisas devem existir uma vez só no programa: a configuração (todas as telas precisam ver as mesmas regras) e o pool de conexões (abrir um pool por tela esgotaria o banco).

Solução. A classe controla a própria criação: guarda o único objeto num atributo de classe e devolve sempre ele.

Em Python. O `__new__` é o método que **cria** o objeto (antes do `__init__`, que só preenche). Sobrescrevendo o `__new__`, a primeira chamada cria e guarda em `_instancia`, e as seguintes devolvem o que foi guardado. Em Java, seria o construtor privado com `getInstance()`.



backend/driftor/config.py — linhas 29-37

```python
class Configuracao:
    _instancia: Configuracao | None = None

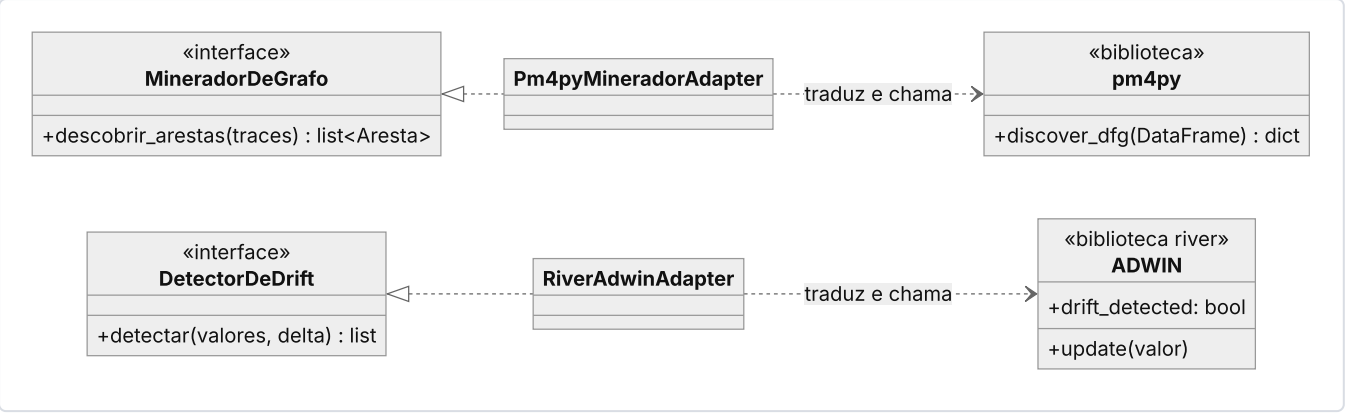
    def __new__(cls) -> Configuracao:
        # Só cria o objeto na primeira chamada; nas outras devolve o mesmo.
        if cls._instancia is None:
            cls._instancia = super().__new__(cls)
            cls._instancia._carregar()
        return cls._instancia
```

Quem usa: `exigir()` (todas as rotas), o menu (`base.html`) e o próprio `BancoDeDados`. O segundo exemplo, `BancoDeDados`, está no capítulo 4. Prova: o teste `test_singletons_devolvem_o_mesmo_objeto` confere `Configuracao() is Configuracao()`.

7.2 Adapter

Problema. As bibliotecas PM4Py e river têm interfaces que não combinam com o nosso domínio. Se as rotas chamassem as bibliotecas direto, trocar uma delas exigiria mexer em todo lugar.

Solução. O programa define a interface que **ele** quer (o *alvo*). Um *adaptador* implementa essa interface e, por dentro, chama a biblioteca (o *adaptado*), traduzindo a entrada e a saída. O cliente (a rota) só conhece o alvo.



Exemplo 1: PM4Py. O app passa `Trace` e quer `Aresta`. O PM4Py quer uma tabela do pandas e devolve `{(origem, destino): frequência}`.

backend/driftor/mineracao/grafo.py — linhas 25–48

```python
class Pm4pyMineradorAdapter(MineradorDeGrafo):
    """Adaptador: implementa a interface do app usando o PM4Py por dentro."""

    def descobrir_arestas(self, traces: list[Trace]) -> list[Aresta]:
        import pm4py  # biblioteca adaptada (o "adaptado")

        # 1. Traduz Traces do domínio para a tabela que o PM4Py entende.
        linhas = [
            {
                "case_id": trace.case_id,
                "atividade": evento.nome_no_grafo(),
                "momento": pd.Timestamp("2000-01-01", tz="UTC") + pd.Timedelta(seconds=posicao),
            }
            for trace in traces
            for posicao, evento in enumerate(trace.eventos)
        ]
        if not linhas:
            return []
        # 2. Chama o PM4Py: grafo de sequência direta (DFG).
        dfg, _inicios, _fins = pm4py.discover_dfg(
            pd.DataFrame(linhas), case_id_key="case_id", activity_key="atividade",
timestamp_key="momento"
        )
        # 3. Traduz a resposta do PM4Py de volta para Arestas do domínio.
        return [Aresta(origem=o, destino=d, frequencia=int(n)) for (o, d), n in dfg.items()]
```

Exemplo 2: ADWIN. O app passa a série inteira e quer a lista de desvios. O ADWIN recebe um valor por vez e liga uma flag.

backend/driftor/mineracao/detector.py — linhas 16-36

```python
class DetectorDeDrift(ABC):
    """Interface que o app espera (o "alvo" do Adapter)."""

    @abstractmethod
    def detectar(self, valores: list[float], delta: float) -> list[tuple[int, int]]: ...


class RiverAdwinAdapter(DetectorDeDrift):
    """Adaptador: implementa a interface do app usando o ADWIN do river por dentro."""

    def detectar(self, valores: list[float], delta: float) -> list[tuple[int, int]]:
        adwin = ADWIN(delta=delta)  # biblioteca adaptada (o "adaptado")
        desvios: list[tuple[int, int]] = []
        inicio_do_trecho = 0
        for indice, valor in enumerate(valores):
            adwin.update(valor)
            if adwin.drift_detected:
                inicio = estimar_inicio(valores, indice, int(adwin.width), inicio_do_trecho)
                desvios.append((indice, inicio))
                inicio_do_trecho = indice + 1
        return desvios
```

Onde o cliente recebe o adaptador. A rota declara que precisa de um `DetectorDeDrift` (a interface), e uma função entrega o adaptador concreto. Trocar a biblioteca é trocar só essa função e o adaptador.

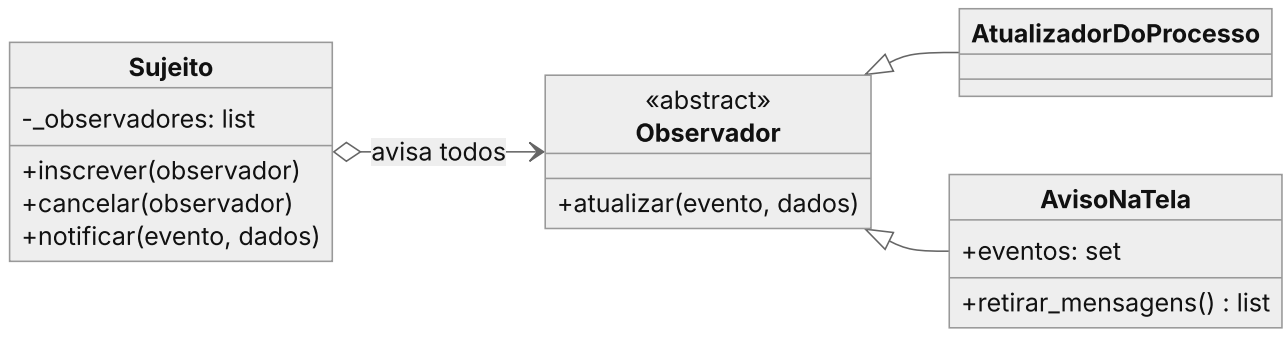
backend/driftor/analises/rotas.py — linhas 19-20

```python
def detector() -> DetectorDeDrift:
    return RiverAdwinAdapter()
```

7.3 Observer

Problema. Quando algo acontece (uma análise termina, uma manutenção é disparada), várias partes querem reagir. Se a rota chamasse cada uma diretamente, toda reação nova exigiria mexer na rota.

Solução. O *sujeito* mantém uma lista de *observadores* e chama `atualizar` em todos quando um evento acontece. Quem publica não sabe quem está ouvindo. Para acrescentar uma reação, basta **inscrever** mais um observador.



backend/driftor/observador.py — linhas 14-58

```python
class Observador(ABC):
    @abstractmethod
    def atualizar(self, evento: str, dados: dict) -> None: ...


class Sujeito:
    def __init__(self) -> None:
        self._observadores: list[Observador] = []

    def inscrever(self, observador: Observador) -> None:
        if observador not in self._observadores:
            self._observadores.append(observador)

    def cancelar(self, observador: Observador) -> None:
        self._observadores.remove(observador)

    def notificar(self, evento: str, dados: dict) -> None:
        for observador in self._observadores:
            observador.atualizar(evento, dados)


class AvisoNaTela(Observador):
    """Guarda o aviso que aparece no topo da próxima tela de um papel (gestor ou técnico).

    Cada caixa só aceita os eventos que interessam ao seu papel.
    """

    def __init__(self, eventos: set[str]) -> None:
        self.eventos = eventos
        self._mensagens: deque[str] = deque(maxlen=5)

    def atualizar(self, evento: str, dados: dict) -> None:
        if evento in self.eventos:
            self._mensagens.append(dados["mensagem"])

    def retirar_mensagens(self) -> list[str]:
        mensagens = list(self._mensagens)
        self._mensagens.clear()
        return mensagens


avisos_por_papel = {
    "gestor": AvisoNaTela({"analise_concluida", "manutencao_concluida"}),
    "tecnico": AvisoNaTela({"manutencao_disparada"}),
}
```

As inscrições ficam num lugar só, em `main.py`:

backend/driftor/main.py — linhas 36-40

```python
# OBSERVER: quem reage a cada evento é decidido aqui, não dentro de quem publica.
publicador_de_analise.inscrever(AtualizadorDoProcesso())
publicador_de_analise.inscrever(avisos_por_papel["gestor"])
publicador_de_manutencao.inscrever(avisos_por_papel["tecnico"])  # recebe "manutencao_disparada"
publicador_de_manutencao.inscrever(avisos_por_papel["gestor"])   # recebe "manutencao_concluida"
```

Exemplo 1: análise concluída. Sujeito `publicador_de_analise`. Observadores: `AtualizadorDoProcesso` (atualiza `atualizado_em` do processo, que sobe na lista) e o `AvisoNaTela` do gestor (o aviso amarelo).

backend/driftor/analises/eventos.py — linhas 15-22

```python
class AtualizadorDoProcesso(Observador):
    """Marca o processo como recém-movimentado, para ele subir na lista de processos."""

    def atualizar(self, evento: str, dados: dict) -> None:
        if evento == "analise_concluida":
            BancoDeDados().executar(
                "update processos set atualizado_em = now() where id = :id",
id=dados["processo_id"]
            )
```

Exemplo 2: manutenção. Sujeito `publicador_de_manutencao`. Ao disparar, publica `manutencao_disparada`, que só a caixa do técnico aceita. Ao concluir, publica `manutencao_concluida`, que só a caixa do gestor aceita. O aviso aparece na **próxima tela** daquele papel, porque `tela()` retira as mensagens da caixa do papel atual:

backend/driftor/telas.py — linhas 33-41

```python
def tela(request: Request, template: str, **dados):
    """Renderiza um template com a configuração, o papel e os avisos do Observer para este
papel."""
    papel = papel_atual(request)
    avisos = avisos_por_papel[papel].retirar_mensagens() if papel else []
    return templates.TemplateResponse(
        request,
        template,
        {"config": Configuracao(), "papel": papel, "papeis": PAPEIS, "avisos": avisos, **dados},
    )
```

Limite honesto. Os avisos ficam na memória do servidor. Se o servidor reiniciar, os avisos ainda não vistos se perdem (os dados no banco não). Para a demonstração isso basta; num produto real, os avisos iriam para uma tabela.

7.4 E o Template Method?

Não faz parte da nossa escolha (Opção 02), mas pode aparecer na conversa. O `RepositorioBase` tem um pedaço parecido: os métodos `listar` e `obter` da base chamam `criar_objeto`, que cada filha implementa do seu jeito. A estrutura geral fica na base, e o passo variável fica na filha. A diferença é que a classe foi criada para eliminar repetição (compactação), e não como um dos três padrões apresentados.

8. Os motores

8.1 Leitura do log

`mineracao/leitor_log.py` transforma o arquivo em `Trace` e `Evento`. Para XES, usa o PM4Py, que junta os pares *start/complete* de cada atividade num evento só, com duração. Para CSV, lê as colunas `Case;Atividade;Inicio;Fim;Tempo(s)`. Em seguida, `Operacao.contar` conta quantas vezes cada rótulo aparece e em quantos traces, e o repositório grava eventos e operações (`substituir_log`).

8.2 Mineração do processo: grafo e variantes

- **DFG (Directly-Follows Graph).** Para cada par de atividades consecutivas num trace, conta quantas vezes "A é seguida diretamente por B". O PM4Py calcula isso (`discover_dfg`), e o `Grafo.montar` junta as arestas com os nós (frequência e duração média). O grafo vira texto Mermaid (`para_mermaid`) e é desenhado na tela.
- **Variantes.** `Variante.agrupar` junta os traces que têm a mesma sequência de atividades e conta quantos traces seguem cada sequência.

8.3 Detecção de desvios: ADWIN

A série. `PontoDaSerie.serie_de_duracao` ordena os traces pelo início e pega, de cada um, a duração da atividade escolhida (ou do trace inteiro). Fica uma lista de números, um por trace.

O tratamento. *Bruto* usa a série como veio. *Tratado* aplica uma média móvel (`suavizar`) que reduz o ruído antes do ADWIN.

O ADWIN (ADaptive WINdowing). Mantém uma janela com os valores recentes. A cada valor novo, testa se existe um corte na janela em que a média da parte antiga e a média da parte nova são diferentes demais para ser acaso. Se existe, sinaliza um desvio e descarta a parte antiga. O **δ** controla a confiança do teste: quanto **menor** o δ, mais forte precisa ser a evidência, e menos desvios aparecem. Usamos 0,002, como no artigo.

O início da anomalia. O ADWIN avisa com atraso: só percebe a mudança alguns traces depois. O `estimar_inicio` procura, antes do aviso, o ponto que melhor divide a série em dois níveis (menor soma dos erros quadráticos). É esse ponto que aparece como "início" no gráfico.

O resultado. `Analise.registrar_resultado` calcula, para cada desvio, a média antes e depois e a variação percentual, e cria os objetos `Drift`.

backend/driftor/dominio.py — linhas 309-321

```
    def registrar_resultado(self, valores: list[float], inicios: list[tuple[int, int]]) -> None:
        """Recebe (índice da detecção, início da anomalia) de cada desvio e calcula as médias."""
        self.total_observacoes = len(valores)
        limites = [0, *[inicio for _, inicio in inicios], len(valores)]
        self.drifts = []
        for posicao, (indice, inicio) in enumerate(inicios, start=1):
            antes = valores[limites[posicao - 1]:inicio] or valores[:1]
            depois = valores[inicio:limites[posicao + 1]] or valores[-1:]
            media_antes, media_depois = statistics.fmean(antes), statistics.fmean(depois)
            variacao = (media_depois - media_antes) / abs(media_antes) * 100 if media_antes else
0.0
            self.drifts.append(
                Drift(novo_id(), posicao, indice, inicio, round(media_antes, 3),
round(media_depois, 3), round(variacao, 2))
            )
```

9. A variabilidade

Conceitos de Linha de Produto de Software (LPS)

- **Variabilidade:** a capacidade de um mesmo código gerar produtos (ou comportamentos) diferentes.
- **Ponto de variação:** o lugar do código em que a diferença acontece. No nosso caso, cada `exigir(...)` nas rotas e o menu.
- **Variante:** cada opção possível nesse ponto. No nosso caso, os papéis Gestor e Técnico.
- **Tempo de ligação (*binding time*):** quando a variante é escolhida. A nossa é escolhida **em tempo de execução**, quando a pessoa entra e escolhe o perfil.
- **Mecanismo: configuração**. As regras ficam num dicionário (`PAPEIS`), e não espalhadas em `if` pelo código.

Como planejamos

O enunciado pede **um** exemplo (2 pontos). O nosso exemplo é a **variabilidade por papel**. O mesmo código atende dois perfis, com telas, menus, permissões e avisos diferentes:

	Gestor	Técnico de manutenção
Começa em	Processos	Manutenções
Processo e Atividade	tudo	sem acesso (403)
Análise	tudo	só o detalhe
Manutenção	dispara, edita, exclui	conclui com observações
Avisos	análise e manutenção concluídas	manutenção nova

Onde está no código

1. Definida no dicionário `PAPEIS`. Mudar o que um papel pode é mudar só este dicionário; acrescentar um papel é acrescentar uma entrada.

backend/driftor/config.py — linhas 13-26

```
13  PAPEIS = {
14      "gestor": {
15          "nome": "Gestor",
16          "descricao": "Mapeia os processos, gera as análises de desvio e dispara as manutenções.",
17          "inicio": "/processos",
18          "pode": {"processos", "atividades", "analises", "ver_analise", "manutencoes",
19  "disparar_manutencao"},
20      },
21      "tecnico": {
22          "nome": "Técnico de manutenção",
23          "descricao": "Recebe as manutenções disparadas, executa e registra as observações.",
24          "inicio": "/manutencoes",
25          "pode": {"manutencoes", "ver_analise", "concluir_manutencao"},
26      },
    }
```

2. Escolhida na entrada, que grava o papel num cookie:

backend/driftor/acesso.py — linhas 28-34

```python
@rotas.post("/entrar")
def entrar(papel: str = Form("")):
    if papel not in PAPEIS:
        raise ErroDeNegocio("Escolha um perfil.")
    resposta = ir_para(PAPEIS[papel]["inicio"])
    resposta.set_cookie("papel", papel, httponly=True, samesite="lax")
    return resposta
```

3. Aplicada pela dependência `exigir`, posta em cada rota. A pergunta é sempre a mesma: `Configuracao().pode(papel, permissao)`.

backend/driftor/telas.py — linhas 49-60

```python
def exigir(permissao: str):
    """VARIABILIDADE: a rota só abre se o papel de quem entrou tem a permissão."""

    def verificar(request: Request) -> str:
        papel = papel_atual(request)
        if papel is None:
            raise PrecisaEntrar()
        if not Configuracao().pode(papel, permissao):
            raise SemPermissao(permissao)
        return papel

    return verificar
```

4. Refletida no menu (só aparece o que o papel pode) e nos botões das telas (por exemplo, "Disparar manutenção" só para quem tem `disparar_manutencao`).

frontend/templates/base.html — linhas 25-31

```
    <nav class="app-nav" aria-label="Principal">
      {% for rota, rotulo, permissao in [("/processos", "Processos", "processos"), ("/atividades",
"Atividades", "atividades"), ("/analises", "Análises", "analises"), ("/manutencoes",
"Manutenções", "manutencoes")] %}
        {% if config.pode(papel, permissao) %}
        <a href="{{ rota }}" {% if atual.startswith(rota) %}aria-current="page"{% endif %}>{{
rotulo }}</a>
        {% endif %}
      {% endfor %}
    </nav>
```

Prova automática: `test_variabilidade_por_papel` entra como técnico e confere que `/processos` e `/analises` dão 403, que ele não consegue disparar manutenção e que, na análise, não aparecem Renomear nem Disparar.

Segurança, honestamente. A entrada é simulada, como pede a demonstração: não há senha, e o papel fica num cookie `httponly`. Num produto real, o papel viria de um login de verdade (por exemplo, o Google via Supabase Auth, como no Deviante). O mecanismo de variabilidade continuaria o mesmo.

10. A técnica de compactação de código

O que é. Reduzir código repetido extraindo o que se repete para **um lugar só** e reaproveitando esse lugar. Não é compactar arquivos (zip) nem minificar. O foco do enunciado é **reuso**.

Nome formal. No catálogo de refatorações de Martin Fowler, o que fizemos no back-end se chama ***Extract Superclass*** (criar uma superclasse comum) e ***Pull Up Method*** (subir para ela os métodos repetidos das filhas). No front-end, é a extração de componentes reutilizáveis (macros).

O que fizemos. Duas extrações, uma no back-end e outra no front-end:

1. **Superclasse genérica (`RepositorioBase`).** Os quatro repositórios tinham os mesmos métodos `listar`, `obter` e `excluir`, mudando só o nome da tabela, e as rotas tinham cada uma a sua função `_xxx_ou_erro`. Tudo isso foi para a base, que as filhas herdam. Cada filha declara só `tabela`, `colunas` e `criar_objeto`, além do SQL que é só dela.
2. **Macros de template (`_componentes.html`).** O botão de excluir com confirmação estava copiado em 7 telas, o campo de renomear em 2 e a linha "nada cadastrado" em 6. Cada um virou uma macro Jinja, importada pelas telas.

	Antes (`1f77872`)	**Depois (`c13a62b`)**
`listar` / `obter` / `excluir`	4 cópias, uma por repositório	1, em `RepositorioBase`
`_xxx_ou_erro` nas rotas	4 funções	1 método herdado (`obter_ou_erro`)
Botão de excluir com confirmação	7 cópias	1 macro (`botao_excluir`)

Comparação no GitHub: `github.com/aland3r/driftor-reuso/compare/1f77872...c13a62b`.

O ganho real é o próximo CRUD. O CRUD de Manutenção foi escrito depois da compactação e já nasceu herdando da base: o repositório dele tem só a tabela, as colunas e o SQL próprio. É a prova de que a técnica serve para reuso.

backend/driftor/manutencoes/repositorio.py — linhas 9–21

```
 9  class RepositorioDeManutencoes(RepositorioBase[Manutencao]):
10      # obter, obter_ou_erro e excluir vêm de RepositorioBase
11      tabela = "manutencoes"
12      colunas = "id, analise_id, drift_id, descricao, status, observacoes, criado_em, concluida_em"
13      nome_no_singular = "manutenção"
14
15      def criar_objeto(self, linha: dict) -> Manutencao:
16          return Manutencao(**{
17              **linha,
18              "id": str(linha["id"]),
19              "analise_id": str(linha["analise_id"]),
20              "drift_id": str(linha["drift_id"]) if linha["drift_id"] else None,
21          })
```

Se perguntarem "mas o total de linhas diminuiu?". Pouco: a base tem 47 linhas, com comentários. O objetivo é que cada comportamento comum exista **uma vez**. Corrigir o `excluir`, por exemplo, agora é mexer em um lugar, e não em quatro.

O vídeo (um por equipe, não listado no YouTube): o roteiro está em `defesa/roteiro-video-compactacao.md`. Ele mostra a repetição no commit antes, a criação da base, uma filha herdando, as macros e os testes passando.

11. Perguntas prováveis

Estrutura e banco

Onde está o Create da Análise? Em `analises/rotas.py`, função `criar`, que responde `POST /processos/{id}/analises`. O SQL fica em `analises/repositorio.py`, método `inserir`.

Onde o sistema conecta com o banco? Em `banco.py`, na classe `BancoDeDados`, que cria o pool com o endereço que vem de `Configuracao().url_do_banco` (variável `DATABASE_URL`).

O clique dispara SQL direto? Não. O clique envia um formulário para uma rota; a rota aplica a regra do domínio e chama o repositório, que executa o SQL pelo `BancoDeDados`.

Vocês usam ORM? Não. O SQL é escrito à mão nos repositórios, com parâmetros nomeados. O SQLAlchemy é usado só para conexão e pool.

Como evitam SQL injection? Os valores vão como parâmetros (`:nome`) e nunca são colados no texto do SQL. O driver envia os valores separados do comando.

Quantas tabelas e quais? Sete: processos, eventos, atividades, operacoes, analises, drifts e manutencoes, em `schema.sql`.

Quando as tabelas são criadas? Ao subir o app, pelo `ao_subir` em `main.py`, que executa o `schema.sql` (`create table if not exists`).

O que acontece ao excluir um processo? O `on delete cascade` apaga os eventos, as operações e as análises dele; e, das análises, os drifts e as manutenções.

Por que não dá para excluir uma atividade? Porque está mapeada em algum processo. A chave `operacoes.atividade_id` tem `on delete restrict`, e o banco recusa.

É MVC? Sim: templates são a view, as rotas são o controller e o domínio com os repositórios é o model. É também organizado em fatias verticais: uma pasta por CRUD.

Classes e OOP

Onde estão as 10 classes? São 13, em `dominio.py`, cada uma com atributos e métodos. Além delas, há as classes dos padrões e dos repositórios.

O que é `@dataclass`? Um decorador que gera o construtor e outros métodos a partir dos atributos declarados.

O que é `self`? O próprio objeto, como o `this` do Java.

Por que `@staticmethod` no `criar`? É uma fábrica: valida os dados e só então cria o objeto. Não precisa de um objeto existente para ser chamada.

Onde tem herança? Os repositórios herdam de `RepositorioBase`; os adaptadores herdam das interfaces abstratas (`MineradorDeGrafo`, `DetectorDeDrift`); os observadores herdam de `Observador`.

Padrões

Quais padrões e por quê? Singleton, Adapter e Observer, com dois exemplos cada (Opção 02). Um de cada família do GoF, e os três ligados a reuso.

Onde está o Singleton 2? `BancoDeDados`, em `banco.py`. O `__new__` cria o pool na primeira vez e devolve o mesmo objeto nas seguintes.

Por que o banco é Singleton? Para o app inteiro dividir um único pool de conexões. Sem isso, cada repositório abriria o seu.

Qual a diferença entre `__new__` e `__init__`? O `__new__` cria o objeto, e o `__init__` preenche. O Singleton precisa controlar a criação, por isso usa o `__new__`.

Como o Adapter ajuda o reuso? Reaproveitamos bibliotecas prontas (PM4Py e river) sem deixar o resto do código depender delas. Trocar a biblioteca é trocar só o adaptador.

Quem é o alvo, o adaptador e o adaptado? No exemplo 2: alvo `DetectorDeDrift`, adaptador `RiverAdwinAdapter` e adaptado `ADWIN` do river.

Como o técnico fica sabendo da manutenção? Pelo Observer: a rota publica `manutencao_disparada`, a caixa de avisos do técnico (inscrita em `main.py`) guarda a mensagem, e a próxima tela do técnico mostra o aviso.

Como acrescentar uma reação nova, por exemplo, mandar e-mail? Criar uma classe que herda de `Observador`, implementar `atualizar` e inscrevê-la em `main.py`. A rota não muda.

Variabilidade

Onde está a variabilidade? No papel. Definida em `PAPEIS` (`config.py`), escolhida na entrada (`acesso.py`) e aplicada pelo `exigir` (`telas.py`) em cada rota e no menu.

Qual é o ponto de variação e quais são as variantes? O ponto de variação é a checagem de permissão em cada rota e no menu. As variantes são os papéis Gestor e Técnico.

Em que momento a variante é escolhida? Em tempo de execução, quando a pessoa escolhe o perfil.

E se o técnico digitar a URL do gestor? Recebe a tela "Este perfil não tem acesso a esta tela", com status 403.

Como acrescentar um terceiro papel? Acrescentar uma entrada em `PAPEIS` com as permissões. As rotas não mudam.

Compactação

Qual técnica de compactação? *Extract Superclass* e *Pull Up Method* (Fowler): extração do código repetido para uma superclasse genérica (`RepositorioBase`) e para macros de template (`_componentes.html`). Reuso por herança e por importação.

Onde ver o antes e o depois? Commits `1f77872` (antes) e `c13a62b` (depois), com a comparação entre os dois no GitHub.

Motores

O que é um drift? Um ponto em que a média da duração de uma atividade muda de forma significativa ao longo dos traces.

O que o δ faz? Controla a confiança do ADWIN. δ menor exige mais evidência e encontra menos desvios.

Por que bruto e tratado dão resultados diferentes? O tratado aplica uma média móvel antes, que tira ruído e muda onde o ADWIN percebe as mudanças.

O que é o grafo? O DFG: quantas vezes cada atividade é seguida diretamente por outra, calculado pelo PM4Py.

12. Checklist para sexta

- ☐ Os dois conseguem abrir, sem ajuda, a linha de cada CRUD × operação (16 combinações) usando `defesa/cruds.md`.
- ☐ Os dois explicam os 3 padrões com a ideia, o desenho e o código, sem ler.
- ☐ Os dois explicam o caminho clique → rota → domínio → repositório → banco com o exemplo de "Disparar manutenção".
- ☐ Os dois sabem onde está a variabilidade (`PAPEIS`, `exigir`, menu) e o que é ponto de variação.
- ☐ Os dois sabem explicar a compactação e mostrar os commits antes e depois.
- ☐ O site está no ar (`driftor-reuso.fly.dev`) e o `DR_MS_ST_20.xes` está à mão para a demonstração.
- ☐ O vídeo de compactação está no YouTube como não listado, e o link está no README.
- ☐ Treino em dupla com as perguntas do capítulo 11, invertendo os papéis.